RealFact: OpenMP-Accelerated, Independence-Aware Real-Time News Verification

Adithya Shyam D (241IT005), Krishna (241IT039), P Suman (241IT049), Saurabh (241IT072)
Department of Information Technology, National Institute of Technology Karnataka, Surathkal, India
IT 301 Parallel Computing, course project, November 2026

Abstract—Evidence-based fact-checking over live news is a large similarity-search problem, and it has a counting problem: outlets republish the same wire story, so ten retrieved "confirmations" may be one report. We present RealFact, a working system that verifies a claim against a continuously refreshed corpus of 42 RSS feeds, optionally extended by on-demand Google News and Wikipedia search. RealFact retrieves a wide candidate pool, clusters near-duplicate copies, weights a cluster of n copies as $1+\lambda \log_2 n$ votes, and returns one of four cited verdicts: Supported, Contradicted, Conflicting Evidence or Unverified. Three stages run as C++/OpenMP kernels. An exact top-M retrieval scan reaches 4.61× over a non-OpenMP serial build at 500K articles; after an instruction-level-parallelism rewrite it becomes bandwidth-bound at 96 GB/s with 4 threads. A corpus-wide all-pairs duplicate detector, which is compute-bound and has a triangular iteration space, reaches 6.92× on 11 heterogeneous cores with dynamic scheduling against 4.98× for static. A block-speculative parallelisation of the sequential leader-clustering algorithm reaches 5.8× with output identical to the sequential algorithm. A 14-check suite verifies that every parallel configuration is bit-identical to the serial build. In a controlled syndication test, naive counting flips a contested claim to Contradicted while RealFact keeps Conflicting Evidence. On a FEVER subset of 179 claims against 25,324 Wikipedia pages, RealFact reaches 81.0% accuracy with settings chosen on a disjoint tuning split.

Index Terms—fact-checking, OpenMP, parallel computing, similarity search, near-duplicate detection, natural language inference, load balancing, roofline model

I. Introduction

Automated fact-checking asks whether a claim is supported by the available evidence. Two weaknesses of common approaches motivate this work. First, style-based "fake news" classifiers judge wording, not truth, and fail when a false claim imitates credible reporting. Second, evidence-based systems built on benchmarks such as FEVER [2], LIAR [3] and HoVer [5] are evaluated on accuracy alone. Their latency and multicore scalability are rarely measured, although both matter when the evidence pool is live and large.

A third problem appears when the evidence is real news. Outlets republish the same wire report, so a ranked list of evidence can be ten copies of one story. If every copy counts as an independent confirmation, one report can outvote several genuinely independent ones. A search engine does not solve this: it returns ranked links, often many copies of the same story, and leaves the user to decide what they say.

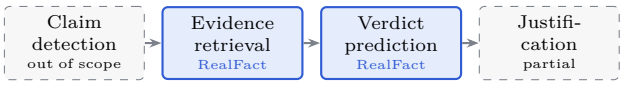


Fig. 1. The fact-checking framework of [1]; RealFact covers the highlighted stages.

Research question. Can large-scale, multi-source evidence retrieval for real-time news verification run efficiently on a multicore CPU using OpenMP, without counting duplicate or syndicated reports as independent evidence?

Contributions.

1) An independence-aware aggregation stage: retrieve wide, cluster near-duplicates, keep the top-K clusters, weight each by $w_c = 1 + \lambda \log_2 n_c$, and issue a cited four-way verdict (Section III).
2) Three OpenMP kernels with deterministic, bit-exact output: an exact top-M scan, a corpus-wide all-pairs duplicate detector, and a block-speculative parallel leader clustering (Section IV).
3) A measurement study using course tools: strong and weak scaling, Karp–Flatt, per-thread load balance on heterogeneous cores, chunk-size sweeps, the roofline model, and Wilcoxon tests (Section VI).
4) Operation on live evidence: 42 permitted RSS feeds refreshed every 20 minutes, plus on-demand web search for the claim being checked, with a saved verification history.
5) An evaluation on LIAR and on a FEVER subset with a held-out tuning split, and a web interface that runs the whole pipeline and shows how existing aggregation rules would judge the same evidence.

II. Background and Related Work

A. The fact-checking framework

Our primary reference is the survey by Guo, Schlichtkrull and Vlachos [1]. It decomposes automated fact-checking into claim detection, evidence retrieval, verdict prediction and justification production. Fig. 1 places RealFact in that framework. Claim detection is out of scope (the user supplies the claim); justification is partial (evidence is cited with each cluster's weight and contribution, but no free-text explanation is generated).

TABLE I
Survey lessons and RealFact's response

Survey point (paraphrased)	RealFact decision
Retrieval then verdict prediction	Separate, separately timed stages; verdicts come from evidence only.
Claim-only datasets invite surface cues	Every verdict lists its evidence; no evidence gives Unverified.
Conflicting evidence is open	Conflicting Evidence is a verdict; syndicated copies are discounted.
NEI is a legitimate outcome	Sufficiency threshold θ and a relevance floor.
Dataset artefacts, evidence leakage	Several datasets; an exclusion mask removes a claim's own article.
Static snapshots vs. recent claims	Live RSS ingestion and on-demand web search.
Efficiency rarely reported	OpenMP kernels measured and checked bit-for-bit against serial.

B. Lessons from the survey

Table I lists the survey points that shaped the design. The most important are that verdicts should be computed from evidence rather than claim wording; that conflicting and untrustworthy evidence is an open problem, since most systems assume a single trustworthy source; that "not enough information" is a legitimate outcome; and that benchmarks are static while real claims concern recent events.

C. Existing approaches

FEVER [2] established retrieve-then-classify over Wikipedia with SUPPORTED/REFUTED/NOT ENOUGH INFO labels. LIAR [3] has 12,836 political statements on a six-point scale but no evidence documents. FakeNewsNet [4] provides full articles with source URLs. HoVer [5] extends retrieval to multi-hop reasoning. Table II compares them with RealFact. None of them handles syndicated duplicates or reports parallel performance.

III. System Design

Fig. 2 shows the architecture. The index side ingests evidence into a contiguous $N \times 384$ float32 embedding matrix. The verify side retrieves, clusters, classifies and aggregates. A separate benchmarking arm times the kernels only, so the systems claim and the accuracy claim can fail independently.

A. Evidence sources

Live RSS. 42 permitted feeds are fetched concurrently every 20 minutes. Only the headline and lead are kept; article pages are never scraped. HTML and boilerplate are stripped, non-English items dropped, and each item is hashed on (source domain, normalised headline), an exact-duplicate filter applied before embedding. At the time of writing the corpus holds 3,770 articles from 40 outlets.

Algorithm 1 Retrieve-wide leader clustering

Require: candidates $V_1..V_m$ best-first, threshold τ, budget K
$L \leftarrow [\,]$ ▷ leaders, in founding order
for $j = 1$ to m do
  if $|L| = K$ then break
  end if
  $b \leftarrow \arg\max_{\ell \in L} V_\ell^\top V_j$ ▷ first maximum
  if $L \neq \emptyset$ and $V_b^\top V_j > \tau$ then assign j to b
  else append j to L (new cluster)
  end if
end for
for remaining j do assign j to its best leader if $> \tau$, else none
end for

On-demand web search. When enabled, the claim is turned into a keyword query (fact-check prefixes, a leading auxiliary verb and punctuation are removed) and two sources are queried in parallel: up to 12 Google News RSS search results and the lead paragraphs of up to 3 Wikipedia pages through the official MediaWiki API. New items pass through the same hash, are embedded and appended to the corpus with origin web. They are then ordinary evidence: they go through the scan, clustering and weighting like any RSS article. Search engines tend to return several outlets carrying the same wire story, so this source makes duplicate handling more important, not less. A search adds about 1–3 s, mostly network time.

FEVER demo corpus. For controlled evaluation, a second corpus holds 25,324 pages of the 2017 FEVER Wikipedia dump (Section VI-G).

B. Retrieval

All vectors are unit-normalised, so cosine similarity is a dot product, $s_i = q^\top v_i$. The scan is exhaustive and exact. We deliberately avoid approximate nearest-neighbour indexes, which would change what is retrieved between configurations and mix a recall change into the speedup. Candidates below a relevance floor (cosine 0.68 for news) are not evidence: NLI models label off-topic premises as contradictions, and for the claim "The Earth is flat" an article about a Martian meteorite was labelled CONTRADICT with $p = 1.00$ before the floor was introduced.

C. Retrieve-wide duplicate clustering

The scan returns the top $M = 10{,}000$ candidates ($M \gg K = 10$). These are clustered by single-pass leader clustering in score order (Algorithm 1); only then are the top-K clusters kept, each represented by its best-scoring member. The order matters: clustering after truncating to K lets copies of one story fill all K slots. In the run shown in Fig. 10, truncate-then-cluster would have kept 4 independent stories and wasted 6 of 10 slots.

D. Independence-weighted aggregation

A three-way NLI cross-encoder labels each cluster representative against the claim (evidence as premise, claim as hypothesis), giving a stance $\sigma_c \in \{+1, -1, 0\}$ and

TABLE II
Comparison with existing approaches (last row: this work)

System	Evidence	Up to date	Retrieval	Duplicate handling	Verdicts	Conflicting evidence	Parallel performance
Style classifiers	None (writing style)	—	None	—	Binary	No	No
LIAR baselines [3]	Claim + metadata	Static	None	—	6-way	No	No
FEVER pipeline [2]	Wikipedia	Static dump	Document + sentence top-k	None	3-way	Folded into NEI	Accuracy only
HoVer [5]	Wikipedia, multi-hop	Static dump	Iterative multi-hop	None	2–3-way	No	Accuracy only
RealFact	Live multi-outlet news, web search, Wikipedia	Yes (RSS + search)	Exact OpenMP scan, top-M with $M \gg K$	Cluster at τ; $w = 1 + \lambda \log_2 n$	4-way, cited	Yes (first-class)	Yes (3 kernels, speedup, E_p, Karp–Flatt, Wilcoxon)

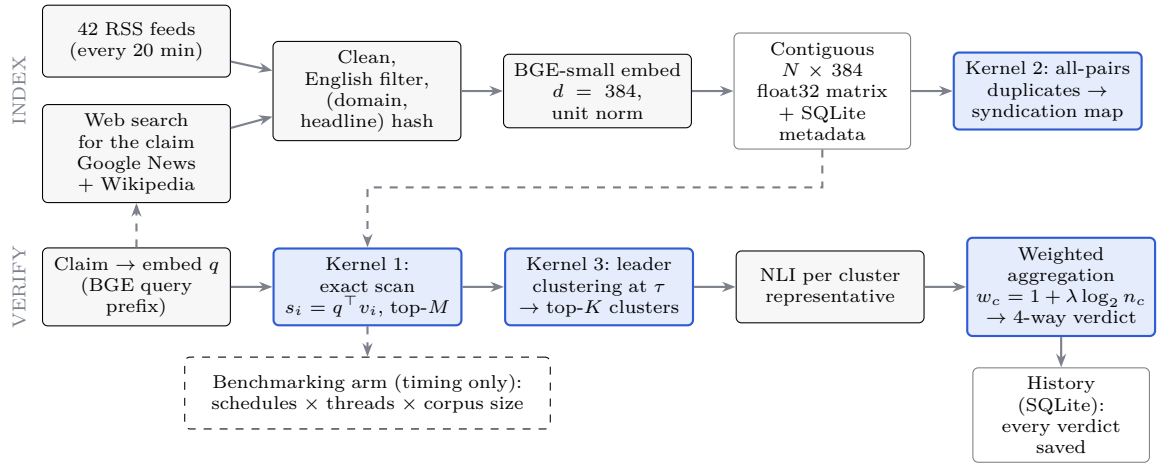


Fig. 2. RealFact architecture. Blue boxes are this project's contributions; the three OpenMP kernels are numbered.

confidence p_c. A cluster of n_c syndicated copies is neither n_c independent witnesses nor exactly one, so its weight interpolates:

$$w_c = 1 + \lambda \log_2 n_c, \qquad \lambda \in [0, 1]. \tag{1}$$

With $\lambda = 0.5$, 16 copies count as 3 votes, not 16 (Fig. 3). The net stance S and the support share D of non-neutral weight are

$$S = \frac{\sum_c w_c p_c \sigma_c}{\sum_c w_c p_c}, \qquad D = \frac{\sum_{\sigma_c = +1} w_c p_c}{\sum_{\sigma_c \neq 0} w_c p_c}. \tag{2}$$

Table III turns S and D into a verdict. Defaults are $\tau = 0.90$, $\lambda = 0.5$, $\theta = 0.6$, $\delta = 0.25$.

E. Verification history

Every verdict is stored with its full result (evidence, weights, parameters, timings) in a separate SQLite database. Past checks can be searched, reopened exactly as they were shown, re-run against today's corpus, or deleted. Syndication stress-test runs are simulations and are not saved.

TABLE III
Four-way verdict rule (checked top to bottom)

Condition	Verdict
$\sum_{\sigma_c \neq 0} w_c p_c < \theta$	Unverified
$\delta < D < 1 - \delta$	Conflicting Evidence
$S \geq 0$ otherwise	Supported
$S < 0$ otherwise	Contradicted

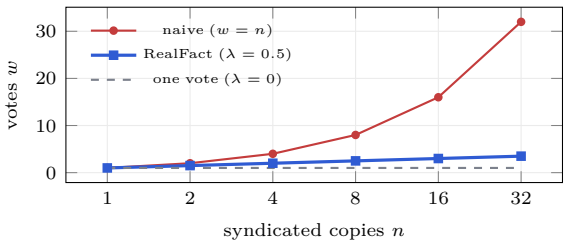


Fig. 3. Cluster weight as a function of the number of syndicated copies.

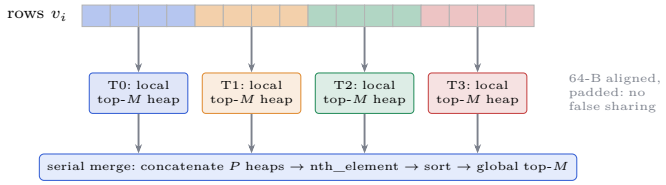


Fig. 4. Kernel 1. Threads scan disjoint row ranges into private, cache-line-padded heaps; a serial merge independent of N produces the global top-M.

Listing 1. Core of kernel 1 (engine/realfact_engine.cpp).

```
struct alignas(64) LocalHeap { std::vector<Hit> h; /* + pad */ };
#pragma omp parallel num_threads(threads)
{
  LocalHeap& mine = heaps[omp_get_thread_num()];
  #pragma omp for schedule(runtime) nowait
  for (int64_t i = 0; i < N; ++i) {
    if (exclude && exclude[i]) continue; // leakage mask
    mine.push(Hit{dot(q, emb + i*d, d), i}, M); // private top-M
  }
} // serial merge: concatenate, nth_element, sort
```

IV. Parallel Implementation

All three kernels use OpenMP [8] and live in one C++ file compiled twice: with OpenMP (librealfact_omp) and without it (librealfact_serial). The serial build is the reference T_1, so OpenMP runtime overhead is not charged to the baseline. Floating-point contraction is disabled (-ffp-contract=off) and every summation order is written out explicitly, so both builds produce bit-identical results, which the test suite checks for every kernel.

A. Kernel 1: exact top-M scan

The scan is a parallel for over corpus rows with schedule(runtime), so the schedule kind and chunk size are run-time parameters (Fig. 4, Listing 1). Each thread keeps a private top-M min-heap aligned and padded to a 64-byte cache line, which prevents false sharing on heap metadata. Ties are broken by row index, making the order total and the output deterministic. The serial merge concatenates the P local heaps and applies nth_element and a sort, $O(PM + M \log M)$, instead of the $O(PM \log M)$ heap merge first planned; at 16 threads and $M = 10{,}000$ this cut the 500K-row query from 11.46 to 9.48 ms.

Dot product. The first version accumulated each row into a single vector register, so each of its 96 vector additions waited for the previous one (a loop-carried dependency) and one core ran at add latency. The second version keeps 16 independent partial sums, which compile to four independent 4-lane NEON registers, reduced by a fixed tree. The order is explicit, so no -ffast-math is needed and the builds stay bit-exact. A register-blocked dot4 scores four vectors per row load and is bit-identical to dot; it powers the multi-query batch kernel and kernels 2 and 3.

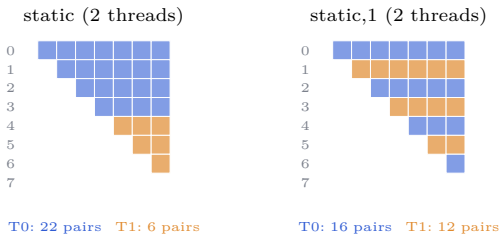


Fig. 5. Kernel 2's triangular iteration space for $N = 8$ rows (cell (i, j) = one pair). Contiguous static ranges give thread 0 3.7× the work of thread 1; cyclic assignment narrows the gap; dynamic scheduling closes it.

B. Kernel 2: all-pairs syndication detection

To measure redundancy across the whole corpus, kernel 2 compares every pair $i < j$ once, links pairs with cosine above τ, and forms connected components with union-find [13]. Rows are processed in blocks of four against each partner row with dot4, so every partner row loaded from memory is used four times; the kernel is compute-bound, unlike the scan. Edges go to per-thread buffers without locks, then are concatenated and sorted, so the output is deterministic. The merge and union-find are serial and timed separately as the Amdahl [10] term.

The iteration space is a triangle: row block b has about $N - 4b$ partners. Fig. 5 shows why the schedule matters. With static and the default chunk, each thread receives one contiguous range of rows, and the first range carries most of the work. Interleaving rows (static,1) or handing them out on demand (dynamic, guided) balances the triangle.

C. Kernel 3: block-speculative leader clustering

Algorithm 1 has a loop-carried dependency: whether candidate j founds a cluster depends on every leader founded before it, so its loop cannot simply be parallelised. Kernel 3 uses block speculation (Fig. 6). For a block of B candidates, the similarity of each candidate to the leaders that exist at block start is computed in parallel. A short sequential walk then compares each candidate only with leaders founded inside the block and makes the decision. Leaders are compared in founding order with a strict "$>$" throughout, so the arg-max and the output are exactly those of the sequential algorithm. Once K leaders exist, assigning the remaining pool is a regular parallel loop. Parallel regions are skipped below 4,096 dot products, where fork/join would cost more than the work.

D. Engineering issues

Apple clang ships no OpenMP runtime, so we link a prebuilt LLVM libomp 19.1.5. PyTorch bundles its own libomp, and if it initialises first our runtime crashes on its first parallel region, so the engine libraries are loaded before any PyTorch import. Benchmarks and ingestion share a lock: an early sweep that overlapped with embedding showed more than 100% CPU utilisation on one thread. macOS exposes no thread-affinity API, so

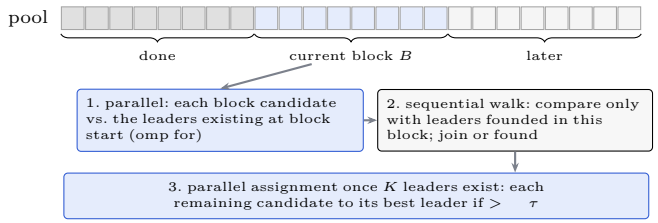


Fig. 6. Kernel 3. Block speculation turns the sequential leader loop into parallel similarity work plus a short sequential decision pass, with output identical to Algorithm 1.

TABLE IV
Platform disclosure

CPU	Apple M3 Pro, 11 cores (5 performance + 6 efficiency), no SMT
Caches	L1d 64 KB; L2 16 MB (P cluster), 4 MB (E cluster)
Memory	18 GB unified
Compiler	Apple clang 21, -O3 -std=c++17 -ffp-contract=off
OpenMP	LLVM libomp 19.1.5; max 11 threads; no pinning on macOS
Models	BGE-small-en-v1.5 [6] (384-d); NLI: nli-deberta-v3-xsmall (news), DeBERTa-v3-base-mnli-fever-anli [7] (FEVER)

OMP_PROC_BIND has no effect; we disclose this rather than claim pinned results.

V. Experimental Setup

Protocol. Kernel 1 is timed under static, dynamic and guided scheduling at 1, 2, 4, 8 and 16 threads on the live corpus and on synthetic corpora of 10K–500K random unit vectors (retrieval cost depends only on $N \times d$), with $M = 10{,}000$. Each configuration runs 2 discarded warm-ups and 11 timed queries; we report medians, IQR and p95, and compare schedules with an exact two-sided Wilcoxon signed-rank test on paired per-query latencies. Kernel 2 is timed under static, static,1, dynamic and guided at 1, 2, 4, 8 and 11 threads on the live corpus and a 10K synthetic corpus, median of 5 runs after a warm-up. Kernel 3 is timed on a 8,983-candidate pool shaped like real syndication (2,000 stories with 1–8 near-copies each). For every configuration of every kernel, output is compared with the serial build.

Roofline expectation [9]. One scan query performs 2 flops per 4-byte value read (0.5 flop/byte), so kernel 1 is memory-bound; its 500K-row working set (768 MB) exceeds every cache. Kernel 2 reuses each row against thousands of partners and should be compute-bound.

VI. Results

A. Kernel 1: strong scaling of the original kernel

Fig. 7 shows speedup and efficiency of the first version (v1) at $N = 500{,}000$. Guided reaches 4.61× at 16 threads (9.48 ms against 43.68 ms serial), static 3.78×, and dynamic with the default chunk of 1 only 1.16×, because it pays an atomic increment per row. Efficiency is 79% at 4 threads and falls beyond, where the kernel

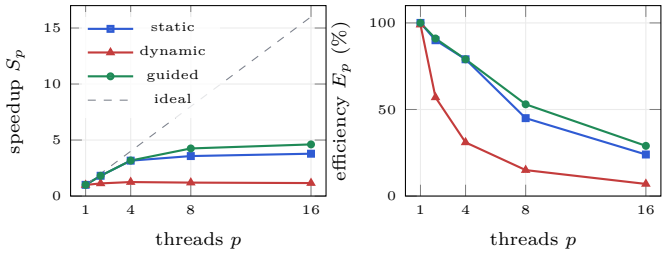


Fig. 7. Kernel 1 (v1) at $N = 500{,}000$, $M = 10{,}000$: speedup and efficiency against the non-OpenMP build ($T_1 = 43.68$ ms).

TABLE V
Kernel 1, v1 to v2, at $N = 500{,}000$ (medians of 11 runs)

	v1	v2	change
Serial T_1	43.68 ms	16.82 ms	2.60× faster
One-core bandwidth	17.6 GB/s	45.7 GB/s	
Best parallel time	9.48 ms	8.04 ms	1.18× faster
Bandwidth at best	81 GB/s	96 GB/s	
Speedup over own T_1	4.61×	2.09×	lower
Threads to 90% of best	16	4	

approaches the memory-bandwidth ceiling (81 GB/s at 16 threads against 17.6 GB/s on one thread). Across 5 corpus sizes × 15 configurations × 11 queries the returned IDs were bit-identical to the serial build in every case.

Guided beats static at 8 and 16 threads (exact Wilcoxon $p = 9.8 \times 10^{-4}$, $r = 0.88$ at 500K, the smallest p attainable with $n = 11$), contrary to the textbook expectation for uniform work. The cause is the heterogeneous CPU: static gives every core an equal share, so threads on efficiency cores finish last. At 1,725 rows the best speedup is 1.7×, since thread start-up costs about as much as the 0.22 ms scan.

B. Kernel 1: course-driven optimisations

ILP rewrite. The 16-accumulator dot product made the serial scan 2.60× faster (43.68 to 16.82 ms; 17.6 to 45.7 GB/s on one core) and the best parallel time 1.18× faster (8.04 ms, 96 GB/s, static at 8 threads). Speedup over the new T_1 fell to 2.09× because the faster core reaches the shared bandwidth ceiling with only 4 threads (Table V): optimising the serial code raised the baseline, which is why speedup must be read next to its baseline.

Karp–Flatt. The serial fraction $e = (1/S_p - 1/p)/(1 - 1/p)$ [12] for v2 static rises from 0.27 at $p = 2$ to 0.49 at $p = 16$. A rising e indicates overhead that grows with p, here contention for memory bandwidth, not a fixed serial section (the only serial code, the heap merge, takes about 0.1 ms).

Load balance. Per-thread busy time measured inside the worksharing loop with padded counters (Table VI) shows static leaving threads idle 11% of the loop at 11 threads, while guided and chunked dynamic let fast cores take up to 1.9× more rows and finish within 1–2% of one another. Dynamic with chunk 1 is balanced but 4.9× slower. A

TABLE VI
Kernel 1 load balance at 11 threads, $N = 500{,}000$

Schedule	ms	max/mean busy	Idle	Rows per thread
static	9.14	1.12	11%	45,454–45,455
dynamic, chunk 1	39.99	1.01	1%	40,489–51,814
dynamic, chunk 4096	8.74	1.02	2%	20,480–69,632
guided	8.15	1.01	1%	31,554–58,761

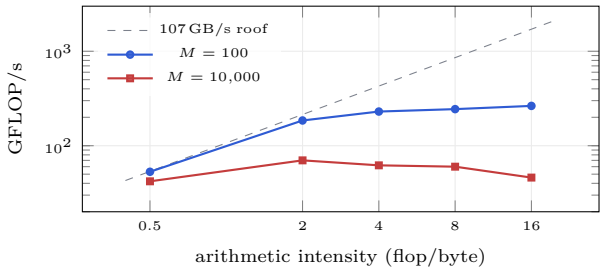


Fig. 8. Roofline of kernel 1 with multi-query blocking ($Q = 1..32$) at 16 threads, $N = 500{,}000$.

chunk sweep showed dynamic needs at least about 256 rows per chunk; guided is best at its default.

Weak scaling. With 31,250 rows per thread, weak efficiency [11] (guided) is 98% at 2 threads, 87% at 4, 51% at 8 and 26% at 16: the same bandwidth ceiling limits weak and strong scaling, so for a bandwidth-bound kernel Gustafson's law offers no escape.

Roofline and multi-query blocking. Scoring Q queries per row load raises arithmetic intensity to $0.5Q$ flop/byte. With $M = 100$, throughput rises from 139 to 688 queries/s (4.9×) and the kernel leaves the bandwidth roof at $Q \approx 4$ (Fig. 8). With $M = 10{,}000$ heap maintenance dominates and the gain is small.

Adaptive threads. Verification now uses $p = \min(\text{cores}, 8, \lceil N/400 \rceil)$ threads with the guided schedule: enough rows per thread to amortise fork/join, and no more threads than the memory system can feed.

C. Kernel 2: all-pairs syndication detection

Fig. 9 and Table VII report kernel 2. The serial build runs at about 48 GFLOP/s. On the live corpus (3,635 articles, 6.6M pairs) dynamic reaches 6.92× at 11 threads (15.4 ms against 106.5 ms), guided 6.69×, static,1 5.56×, and static only 4.98×, because its busiest thread does 1.90× the average work, exactly the triangular imbalance of Fig. 5. The 10K synthetic corpus (50M pairs) shows the same ordering. The serial merge and union-find take under 0.15% of the parallel time, so Amdahl's serial fraction is not the limit. The limit is the hardware: 5 threads on performance cores give 4.6×, and each efficiency core adds roughly 40% of a performance core, so 11 threads are worth about 7 performance cores. Every one of the 40 configurations produced edges and groups identical to the serial build.

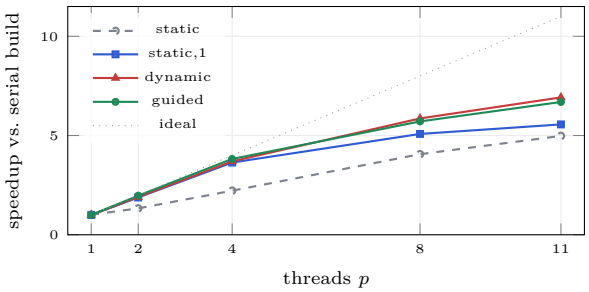


Fig. 9. Kernel 2 on the live corpus (3,635 articles, 6.6M pairs).

TABLE VII
Kernel 2 at 11 threads: speedup and busiest-thread load (max/mean busy time)

	Live, 6.6M pairs			Synthetic, 50M pairs		
Schedule	ms	S_p	max/mean	ms	S_p	max/mean
serial	106.5	1.00	—	809.3	1.00	—
static	21.36	4.98	1.90	168.70	4.80	1.76
static,1	19.16	5.56	1.37	129.88	6.23	1.11
dynamic	15.40	6.92	1.00	126.01	6.42	1.00
guided	15.91	6.69	1.12	127.18	6.36	1.00

TABLE VIII
Kernel 3 on a pool of 8,983 syndicated candidates (median ms)

K	NumPy	serial	2T	4T	11T	vs serial	vs NumPy
10	0.96	1.72	0.89	0.47	0.30	5.8×	3.2×
100	2.04	17.84	9.11	4.70	2.73	6.5×	0.75×
1000	74.29	191.32	99.22	52.71	29.82	6.4×	2.5×

On the live corpus at $\tau = 0.9$, 13.1% of articles have a near-copy, forming 187 groups; the largest is one story carried as 23 articles across 10 outlets (Fig. 12a). Under naive counting that story would count as 23 confirmations; RealFact gives it $1 + 0.5 \log_2 23 = 3.26$ votes.

D. Kernel 3: parallel leader clustering

Table VIII compares kernel 3 with the serial build and with the NumPy reference implementation of Algorithm 1, whose assignment step is a matrix product executed by Apple's Accelerate BLAS. All 15 configurations returned output identical to the reference. At the default $K = 10$, 11 threads are 5.8× faster than the serial build and 3.2× faster than NumPy. At $K = 1000$ the kernel is 2.5× faster than NumPy. At $K = 100$ NumPy is faster (0.75×): its matrix product runs on the M3's dedicated matrix unit, a hardware accelerator rather than more CPU parallelism. On the live corpus the pool above the relevance floor is usually a few dozen articles, so the stage takes microseconds and stays serial by design.

E. Correctness

Table IX lists the 14-check suite, runnable from the command line or the benchmark tab. All checks pass for every kernel version reported here.

TABLE IX
Correctness suite (tests/test_correctness.py); all 14 pass

Check	What it verifies
Scan vs NumPy	45 schedule × thread × chunk configurations return the exact top-M.
Bit-exactness	36 parallel runs identical to the serial build at $M = 10{,}000$.
Ties, edge cases	40-way tie in index order; $M > N$, $N \in \{0, 1\}$, odd d.
Leakage mask	Masked rows are never returned.
Batch, accounting	Batched = single query; per-thread rows sum to N.
Aggregation	S, D and verdicts on hand-computed cases.
Kernel 3	162 configurations identical to the NumPy reference.
Kernel 2	Edges equal a float64 Gram matrix, groups equal BFS, pairs covered exactly once.
Statistics, data	Exact Wilcoxon vs enumeration; ingestion keys; label maps.

TABLE X
Same evidence after syndicating the contradicting article ×10

Aggregation rule	Verdict	S	D	$\sum w$
RealFact ($\lambda = 0.5$)	Conflicting	−0.22	0.31	6.2
One vote per cluster	Conflicting	0.04	0.55	4.0
Naive counting	Contradicted	−0.62	0.10	15.0
Raw top-K	Contradicted	−0.56	0.14	10.0

F. Independence filter: controlled syndication test

The interface can republish one real article as n near-identical copies under labelled simulated outlets, for one query; the copies are appended to the matrix kernel 1 scans. For the claim "Kuwait-flagged tanker with 21 Indian seafarers hit near Strait of Hormuz, crew safe", four independent clusters were retrieved (one SUPPORT, two NEUTRAL, one CONTRADICT) and the verdict was Conflicting Evidence. After republishing the contradicting article ×10 (Table X), naive counting and raw top-K flip to Contradicted, while RealFact counts the 11-copy cluster as 2.73 votes and correctly stays Conflicting Evidence. On unmodified data all rules agree (Fig. 11b), the expected control: cross-outlet reports score 0.75–0.88, below τ, and are independent reports, not copies.

G. Verification quality

LIAR. On the 1,267 labelled test statements against the live corpus, 99.1% came back Unverified and accuracy equalled the majority-class baseline (37.6%). LIAR's claims date from 2007–2016 and the corpus covers a few days of current news, so this run measures corpus coverage, not reasoning: the system correctly declines to guess.

FEVER demo subset. To test reasoning with a matched evidence pool we built a corpus of 25,324 pages from the FEVER 2017 dump: the gold pages of 179 dev claims (60 SUPPORTS, 59 REFUTES, 60 NOT ENOUGH INFO), 2,723 pages cited by other dev claims as hard distractors, and 22,476 random pages. SUPPORTS and REFUTES claims were selected with evidence in a page's first four sentences, because the index embeds page leads. The NLI model [7] is trained on FEVER's train split, so testing on dev claims is fair.

TABLE XI
FEVER demo subset: 179 test claims, 25,324 pages (†settings chosen on the held-out tuning split)

Configuration	Acc.	F1	ms	R@1
xsmall NLI, paragraph, default	58.7%	0.585	65	89.1%
FEVER NLI, paragraph, default	74.3%	0.741	95	89.1%
FEVER NLI, sentence, tuned†	81.0%	0.807	114	92.4%

All tuning used a separate split of 180 other dev claims that share no claim or gold page with the 179 test claims; the test claims were scored once afterwards. Three changes were chosen on that split: sentence-level NLI (each sentence of a page is checked and the most decisive one is used, as FEVER systems select sentences), a relative cutoff keeping clusters within 0.04 cosine of the best match (still up to $K = 10$), and $\theta = 0.8$ with a relevance floor of 0.64. A larger DeBERTa-v3-large NLI model was no better on the tuning split and 2–3× slower. Table XI reports the result: 81.0% accuracy (macro F1 0.807) against 74.3% before tuning, with the gold page ranked first for 92.4% of verifiable claims. The tuning split scores 83.9%, close to the test result, as expected when settings are not fitted to the test claims. Remaining errors are mostly NOT ENOUGH INFO claims labelled Contradicted (15 of 60), whose pages exist and partly conflict with the claim. Being curated for lead-only evidence, the subset is not comparable to published full-FEVER scores.

Latency. A typical live claim takes about 100–125 ms end to end: claim embedding 8–20 ms, kernel 1 0.4 ms, kernel 3 under 0.01 ms, NLI about 90–110 ms, aggregation under 0.1 ms; web search, when enabled, adds 1–3 s. NLI dominates at the current corpus size; kernel 1 becomes significant at hundreds of thousands of articles, where it costs 8–44 ms per query.

VII. User Interface

The single-page interface has six tabs: Verify a claim, Existing vs RealFact, Parallel benchmark, Live corpus, Evaluation and History. Figs. 10, 11 and 12 are screenshots of the running system.

VIII. Threats to Validity and Limitations

- Syndication vs. convergence. Embedding similarity cannot fully separate syndication from independent outlets reporting the same fact, so the filter's effect is an upper bound.
- No labelled live data. Live-news accuracy cannot be measured: no labelled dataset matches current news. LIAR measures coverage; FEVER is a curated, lead-only subset.

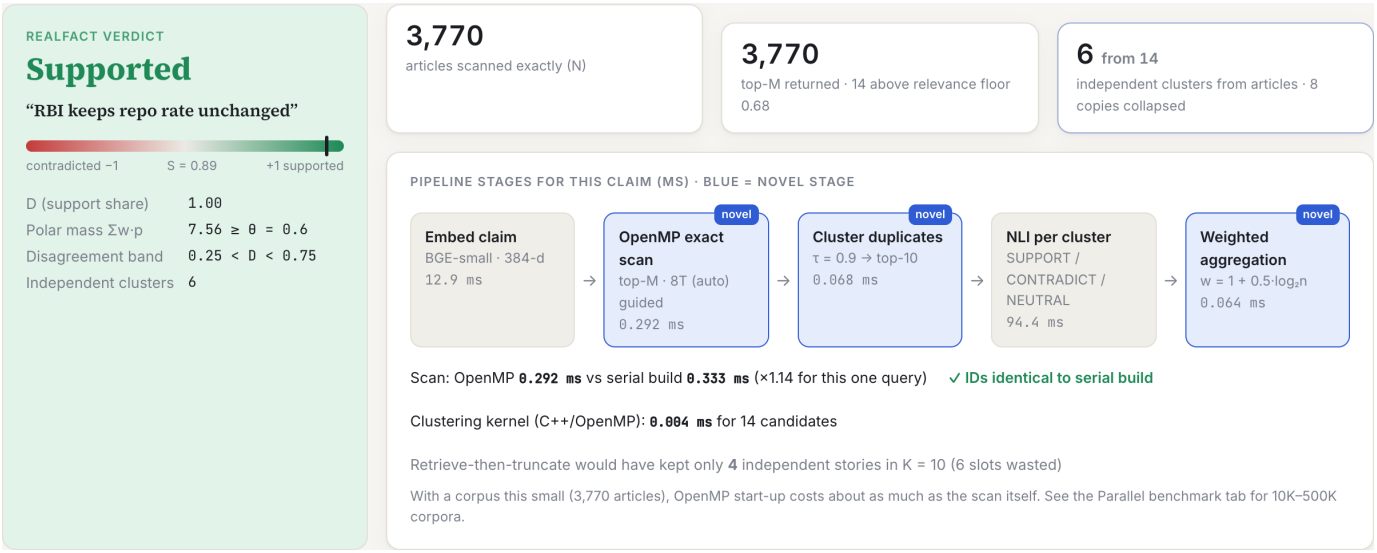
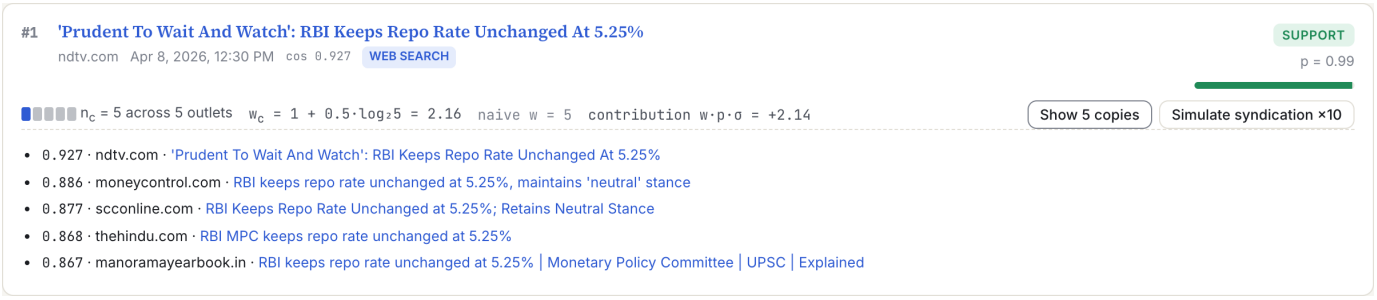
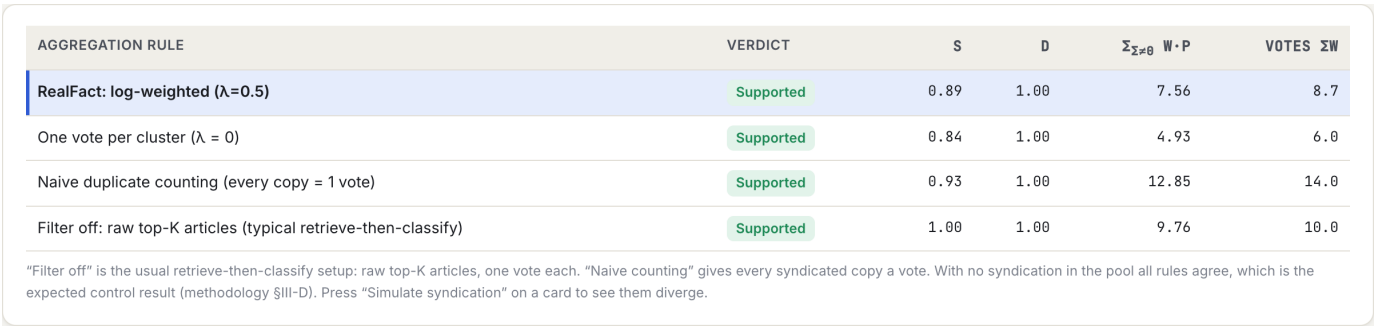


Fig. 10. Verify tab: verdict panel with the S meter and D, polar mass and cluster count; corpus statistics; and the timed pipeline stages (blue = novel stage), with the per-query OpenMP vs. serial comparison and the retrieve-then-truncate counterfactual.



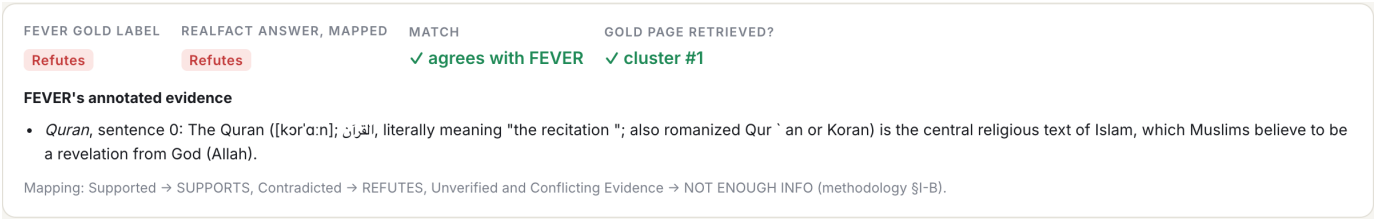
(a) Evidence card: one cluster of five articles from five outlets (the top one fetched by web search); weight $w_c = 2.16$ instead of a naive 5.



AGGREGATION RULE	VERDICT	S	D	$\Sigma_{\Sigma\neq\theta}$ W·P	VOTES ΣW
RealFact: log-weighted (λ=0.5)	Supported	0.89	1.00	7.56	8.7
One vote per cluster (λ = 0)	Supported	0.84	1.00	4.93	6.0
Naive duplicate counting (every copy = 1 vote)	Supported	0.93	1.00	12.85	14.0
Filter off: raw top-K articles (typical retrieve-then-classify)	Supported	1.00	1.00	9.76	10.0

“Filter off” is the usual retrieve-then-classify setup: raw top-K articles, one vote each. “Naive counting” gives every syndicated copy a vote. With no syndication in the pool all rules agree, which is the expected control result (methodology §III-D). Press “Simulate syndication” on a card to see them diverge.

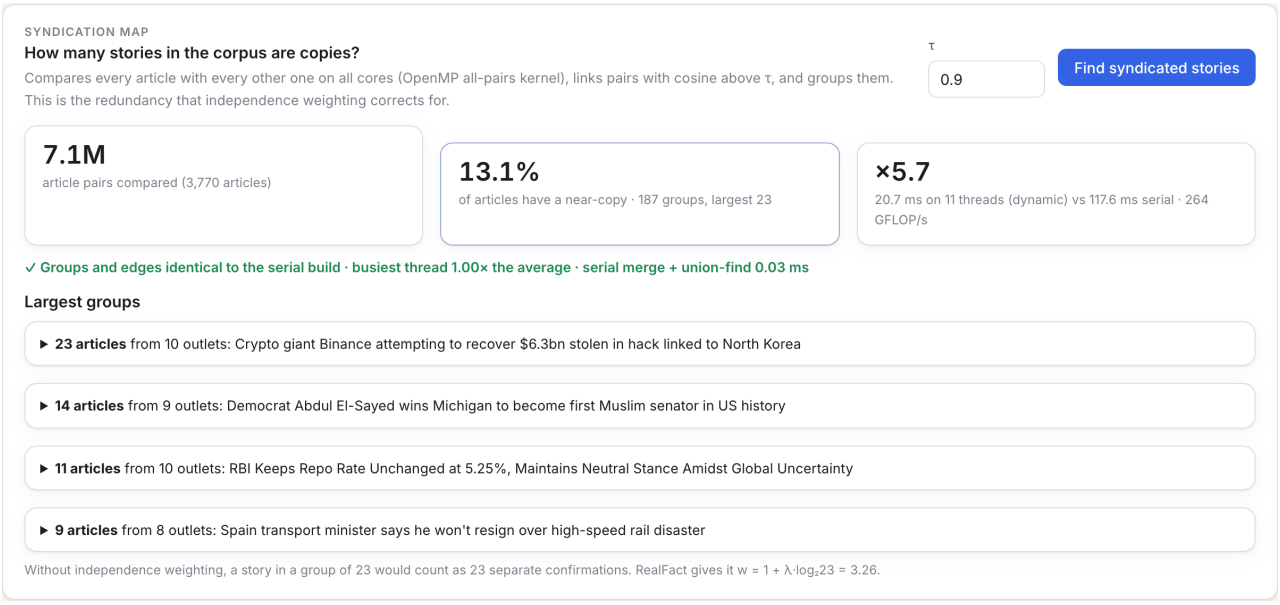
(b) Existing aggregation rules vs. RealFact on the same evidence; without syndication all rules agree, the expected control result.



(c) FEVER demo mode: FEVER’s gold label and annotated evidence next to RealFact’s answer and the gold page’s retrieval rank.

Fig. 11. Result views of the Verify tab.

- NLI behaviour. The relevance floor and relative cutoff are tuned thresholds; topically similar articles about



(a) Syndication map (Live corpus tab), computed by kernel 2: 7.1M pairs on 11 threads, 13.1% of articles have a near-copy, output identical to the serial build.

WHEN	CORPUS	CLAIM	VERDICT	S	CLUSTERS	MS	
Nov 4, 2026, 9:40 PM	FEVER ✓	The Quran is Wiccan.	Contradicted	-1.00	2	336	Open Re-run ×
Nov 4, 2026, 9:40 PM	Live	RBI keeps repo rate unchanged	Supported	0.89	6	108	Open Re-run ×
Nov 4, 2026, 9:34 PM	FEVER ✓	The Quran is Wiccan.	Contradicted	-1.00	2	337	Open Re-run ×
Nov 4, 2026, 9:34 PM	Live	RBI keeps repo rate unchanged	Supported	0.89	6	125	Open Re-run ×
Nov 4, 2026, 9:33 PM	FEVER ✓	The Quran is Wiccan.	Contradicted	-1.00	2	338	Open Re-run ×
Nov 4, 2026, 9:33 PM	Live WEB	RBI keeps repo rate unchanged	Supported	0.89	6	131	Open Re-run ×
Nov 4, 2026, 9:33 PM	FEVER ✓	The Quran is Wiccan.	Contradicted	-1.00	2	282	Open Re-run ×
Nov 4, 2026, 9:32 PM	Live WEB	RBI keeps repo rate unchanged	Supported	0.89	6	100	Open Re-run ×
Nov 4, 2026, 9:26 PM	FEVER ✓	The Quran is Wiccan.	Contradicted	-1.00	2	321	Open Re-run ×
Nov 4, 2026, 9:25 PM	FEVER ×	UNESCO's City of Literature program was launched in May of 2004.	Contradicted	-1.00	1	103	Open Re-run ×

Stored locally in `data/history.sqlite`. Syndication stress-test runs are simulations and aren't saved. ✓/× marks whether a FEVER demo answer matched the gold label.

(b) History tab: saved verifications that can be reopened, re-run or deleted; WEB marks checks that used web search, and ✓/× marks whether a FEVER answer matched the gold label.

Fig. 12. Corpus-level and history views.

different entities can still be labelled CONTRADICT.

- Heterogeneous cores, no pinning. Results reflect the macOS scheduler's placement across performance and efficiency cores and are not directly comparable to pinned results on a homogeneous x86 server.
- Corpus drift. The live corpus grows between runs; every benchmark records the size it ran on.
- Web search terms. The Google News feed is intended for personal, non-commercial use, which covers this academic prototype.

IX. Conclusion and Future Work

RealFact shows that evidence-based news verification over a live, multi-outlet corpus runs in about 100 ms per claim on a laptop CPU, and that counting independent sources rather than copies changes verdicts exactly when syndication is present and leaves them unchanged otherwise. The three OpenMP kernels illustrate three regimes from the course: a memory-bound scan whose scaling stops at the bandwidth roof (where an ILP rewrite and multi-query blocking were the effective levers), a compute-bound triangular all-pairs kernel where the choice of schedule

decides load balance (6.92× dynamic vs. 4.98× static), and an inherently sequential algorithm made parallel by block speculation without changing its output. A 14-check suite kept every optimisation bit-exact against the serial build.

Future work: index the full FEVER dump and FakeNewsNet for accuracy comparable to published baselines; measure on a pinned, homogeneous multi-socket server; use float16 or int8 embeddings to raise the bandwidth ceiling; a heap-free top-M so batching pays off at large M; and generated justifications grounded in the cited clusters.

References

[1] Z. Guo, M. Schlichtkrull, and A. Vlachos, "A survey on automated fact-checking," Trans. Assoc. Comput. Linguistics, vol. 10, pp. 178–206, 2022.

[2] J. Thorne, A. Vlachos, C. Christodoulopoulos, and A. Mittal, "FEVER: A large-scale dataset for fact extraction and VERification," in Proc. NAACL-HLT, 2018, pp. 809–819.

[3] W. Y. Wang, "'Liar, liar pants on fire': A new benchmark dataset for fake news detection," in Proc. ACL, 2017, pp. 422–426.

[4] K. Shu, D. Mahudeswaran, S. Wang, D. Lee, and H. Liu, "FakeNewsNet: A data repository with news content, social context, and spatiotemporal information for studying fake news on social media," Big Data, vol. 8, no. 3, pp. 171–188, 2020.

[5] Y. Jiang, S. Bordia, Z. Zhong, C. Dognin, M. Singh, and M. Bansal, "HoVer: A dataset for many-hop fact extraction and claim verification," in Findings of EMNLP, 2020, pp. 3441–3460.

[6] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, "C-Pack: Packaged resources to advance general Chinese embedding," arXiv:2309.07597, 2023.

[7] M. Laurer, W. van Atteveldt, A. Casas, and K. Welbers, "Less annotating, more classifying: Addressing the data scarcity issue of supervised machine learning with deep transfer learning and BERT-NLI," Political Analysis, vol. 32, no. 1, pp. 84–100, 2024.

[8] OpenMP Architecture Review Board, OpenMP Application Programming Interface, Version 5.2, 2021.

[9] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," Commun. ACM, vol. 52, no. 4, pp. 65–76, 2009.

[10] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in Proc. AFIPS Spring Joint Computer Conf., 1967, pp. 483–485.

[11] J. L. Gustafson, "Reevaluating Amdahl's law," Commun. ACM, vol. 31, no. 5, pp. 532–533, 1988.

[12] A. H. Karp and H. P. Flatt, "Measuring parallel processor performance," Commun. ACM, vol. 33, no. 5, pp. 539–543, 1990.

[13] R. E. Tarjan, "Efficiency of a good but not linear set union algorithm," J. ACM, vol. 22, no. 2, pp. 215–225, 1975.